neighbor(s) of a generic point (possibly not in the dataset itself); in chapter 9, we introduce k-d trees, a data structure specifically invented to solve this problem.

K-d trees are the best solution to date for indexing low to medium dimensional datasets that will completely fit in memory; when, however, either we have to operate on high-dimensional data, or with big datasets that won't fit in memory, then k-d trees are not enough, we will need to use more advanced data structures.

In this chapter we first present a new problem, one that will push our indexing data-structure beyond its limits, and then introduce two new data structures, R-trees and SS-trees, that can help us solve this category of problems efficiently.

Brace yourself, because this is going to be a long journey (and a long chapter!) through some of the most advanced material we have presented so far. But we'll try to make it through this journey step by step, section by section, so don't let the length of this chapter intimidate you!

10.1 Right Where We Left

Let's briefly recap where we left off in previous chapters: we were designing software for an e-commerce company, an application to find the closest warehouse selling a given product, for any point on a very large map: check out figure 9.4 to visualize it. To have a ballpark idea of the kind of scale we need, we want to serve millions of clients per day across the Country, taking products from thousands of warehouses, also spread across the whole map.